

Spring Boot, from zero to Aegis

A code-first guide to the API, SQL migrations,
and the first vertical slice.

MENTAL MODEL

You already know half the system.

React describes the interface. Spring owns the server-side decisions.

React / Next.js

- Component
- State
- Fetch call
- Rendered result

Spring Boot

- Controller
- Service
- Repository
- HTTP response

Aegis rule

The backend decides whether an asset is ready, reservable, and safe to release.

UI state can be temporary. Custody and authorization cannot be.

REALITY CHECK

What the repository actually contains

The architecture is broader than the code that exists today.

Running backend slice

Health · POST /auth/login · GET /auth/me
Java 21 · Spring Boot 4.1.1
JdbcClient · PostgreSQL · Flyway

Planned, not implemented yet

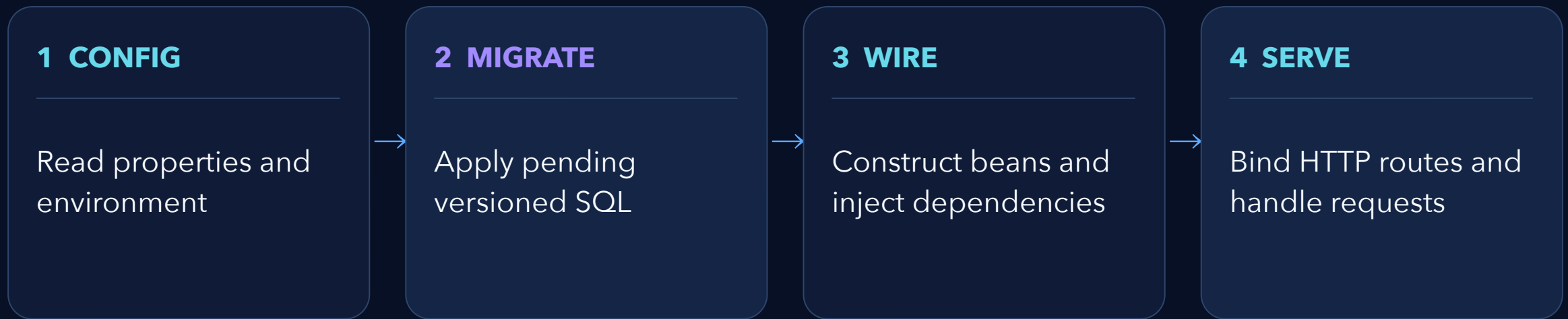
Asset catalogue and readiness · reservations
MQTT command flow · loans and returns
Admin workflows · SwiftUI client

This deck teaches existing code first and labels future architecture as future.

RUNTIME

What happens when the API starts

Spring creates the application; Flyway shapes the database before traffic reaches it.



Flyway is version control for database changes – not a substitute for SQL.

ENTRY POINT

Two annotations and one entry point

A small class can configure a large application because Spring scans and wires the packages below it.

AEGIS CONTROL APPLICATION · CONDENSED

```
@SpringBootApplication
public class AegisControlApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            AegisControlApplication.class, args);
    }
    @Bean
    Clock clock() {
        return Clock.systemUTC();
    }
}
```

@SpringBootApplication

Entry point for configuration, component scanning, and auto-configuration.

@Bean Clock

One reusable UTC time source that Spring can inject into other classes.

DEPENDENCY INJECTION

The container builds the graph

You declare what a class needs; Spring supplies the dependency.

ACCOUNTREPOSITORY.JAVA · EXACT EXCERPT

```
@Repository
class AccountRepository {
    private final JdbcClient jdbc;

    AccountRepository(JdbcClient jdbc) {
        this.jdbc = jdbc;
    }
}
```

Spring**JdbcClient**

injected into

AccountRepository

A bean is an object managed by the Spring container.

ARCHITECTURE

One request crosses four responsibilities

Keep Aegis business decisions in the service/domain, never in the Web or iOS client.



A controller should not become a place to invent readiness or custody rules.

DATA ACCESS

Aegis uses JdbcClient, not JPA

SQL is visible, parameterized, and mapped to a Java record/class.

ACCOUNTREPOSITORY.JAVA · CONDENSED

```
return jdbc.sql(SELECT_ACCOUNT +  
    "WHERE lower(email) = lower(:email)")  
    .param("email", normalizedEmail)  
    .query(AccountRepository::map)  
    .optional();
```

What :email means

A named SQL parameter, supplied separately from the SQL text.

What map does

Converts each ResultSet row into an Account domain object.

SCHEMA EVOLUTION

Flyway gives SQL a version history

Aegis stores migration scripts in the repository; Flyway applies them in version order.

```
services/api/src/main/resources/
```

```
db/migration/
```

```
V001__create_schema_and_extensions.sql
```

```
V002__create_identity_and_catalog.sql
```

Add the next version; do not rewrite V002

1 Versioned files

Every database change has a numbered SQL file.

2 History table

Flyway records applied versions and checksums to prevent drift.

INTEGRITY

The database is the final guardrail

A UI check is helpful. A PostgreSQL constraint still protects the data under concurrency.

V002 SQL · EXACT EXCERPTS

```
CONSTRAINT ck_users_role  
  CHECK (role IN ('ADMIN', 'TECHNICIAN')),  
  
CREATE UNIQUE INDEX ux_users_email_ci  
  ON aegis.users (lower(email));
```

CHECK

Rejects an unsupported role value at write time.

UNIQUE INDEX

Prevents two accounts with the same email ignoring case.

One invariant can appear in UX, service checks, and the database – each layer has a job.

VERIFICATION

How do we know it works?

Use tests at the boundary where each failure would surface.

HTTP behavior

MockMvc exercises status codes, JSON, authentication, and authorization.

Real database

Integration tests run against isolated PostgreSQL rather than an in-memory substitute.

Schema evolution

Flyway tests verify a fresh database and versioned upgrade path.

Compile $\neq$ verified behavior. Inspect the failing boundary, then add the smallest relevant test.

PRACTICE

Your first 20-minute code tour

Open these four real files in order. Explain each one out loud.

01 **AegisControlApplication.java**

Where does Spring start?

02 **application.properties**

Where are database and Flyway configured?

03 **V002__create_identity_and_catalog.sql**

Which rule does PostgreSQL enforce?

04 **AccountRepository.java**

Where does Java issue parameterized SQL?

Next module: trace /auth/login and /auth/me end to end.